Unit 4.2: Interpretable Feature Engineering and Pipelines

DAIR-3 Workshop, Summer 2026

Presented by Suraj Rampure (rampure@umich.edu)

Recap: Birthweights

- At the end of Unit 4.1, you built a 4-feature model that predicts `birthweight` given various other features.

In [2]:

```
births = load_birthweight_1971()
```

In [3]:

```
births.sample(5)
```

Out[3]:

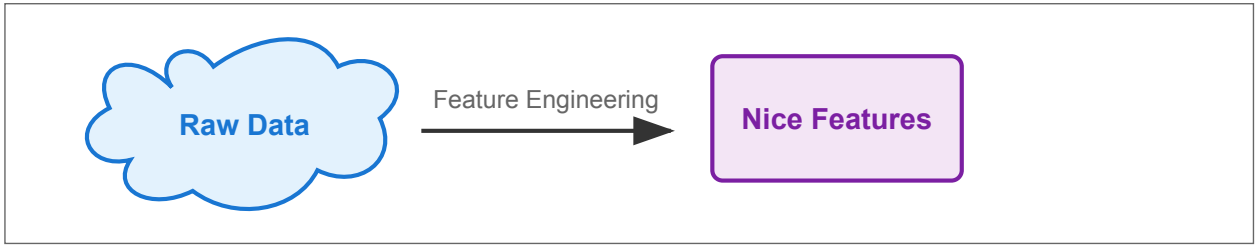
	year	state	county	smsa	sex	dadrace	momrace	momage	birthorder	dadage	birthweight	plurality	interval	popsiz
990194	1971	31	009	094	male	White	White	28	NaN	40.0	2863.0	2	NaN	2
1522461	1971	43	045	000	male	White	White	22	1.0	25.0	3629.0	1	NaN	9
1443991	1971	39	035	187	female	White	White	35	4.0	33.0	3402.0	1	NaN	9
530401	1971	15	082	064	female	White	White	29	4.0	29.0	3260.0	1	17.0	9
577399	1971	16	089	000	male	White	White	22	1.0	23.0	3742.0	1	NaN	9

- What if we'd like to produce **new** features, using our understanding of how the features and target are related?

Feature engineering 

From raw data to useful features

- In supervised learning, we often don't jump straight from raw data to a model.
- First, we create features that reflect the scientific or operational meaning in the data.
- The rest of this unit is about making those transformations interpretable and reproducible.



The goal of feature engineering

- **Feature engineering** is the act of creating useful quantitative features from the data you already have.
- Before writing code, it helps to name the kind of transformation you want.

Starting point	Common feature-engineering move	`sklearn` tool we'll use
Numeric feature	transform or rescale it, e.g. `log(x)`, x^2, or a standardized version	`FunctionTransformer`, `PolynomialFeatures`, `StandardScaler`
Categorical feature	turn categories into 0/1 indicator columns	`OneHotEncoder`
Mixed feature set	apply different transformations to different columns	`ColumnTransformer`
Whole workflow	store preprocessing and model fitting together	`Pipeline` / `make_pipeline`

Pedagogical note

- Our plan in this block is to look at how to reproducibly create features.
- Tomorrow, in Units 4.3 and 4.4, we will look at how to pick which features to use and how to document these design decisions.

Example 1: Numerical transformations

- The following dataset, built into the `seaborn` plotting library, contains various information about older cars.
- Our goal is to predict `mpg` from `horsepower`.
- The feature-engineering idea is to create `log(horsepower)` from `horsepower`, then fit a linear model using that transformed feature.

In [4]:

```
import seaborn as sns

mpg = sns.load_dataset('mpg').dropna()
mpg.head()
```

Out[4]:

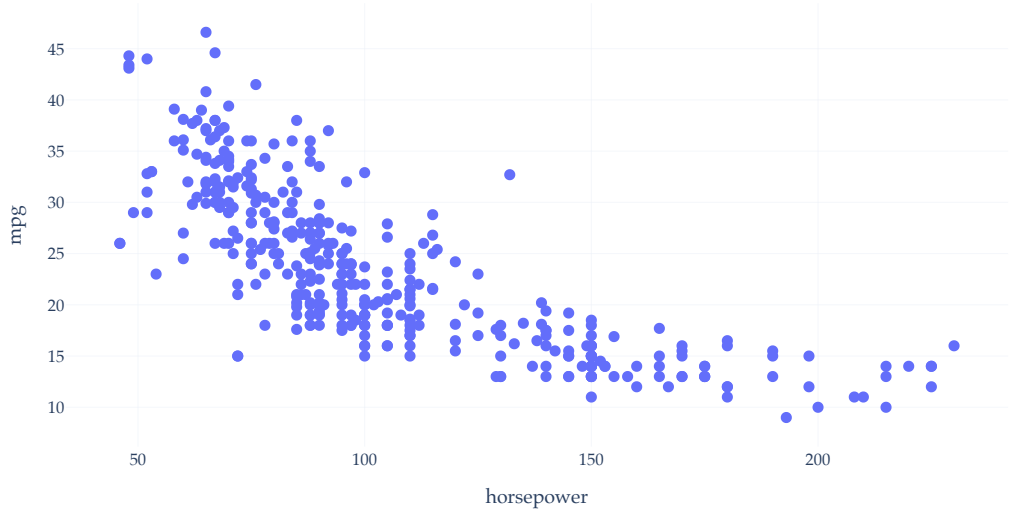
	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693	11.5	70	usa	buick skylark 320
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymouth satellite
3	16.0	8	304.0	150.0	3433	12.0	70	usa	amc rebel sst
4	17.0	8	302.0	140.0	3449	10.5	70	usa	ford torino

The relationship between `horsepower` and `mpg`

- Let's investigate the relationship between `horsepower` and `mpg`.

In [5]:

```
px.scatter(mpg, x='horsepower', y='mpg')
```



- It appears that there is a negative association between `horsepower` and `mpg`, though it's not quite linear.
- Let's try and fit a simple linear model that uses `horsepower` to predict `mpg` and see what happens.

Predicting `mpg` using `horsepower`

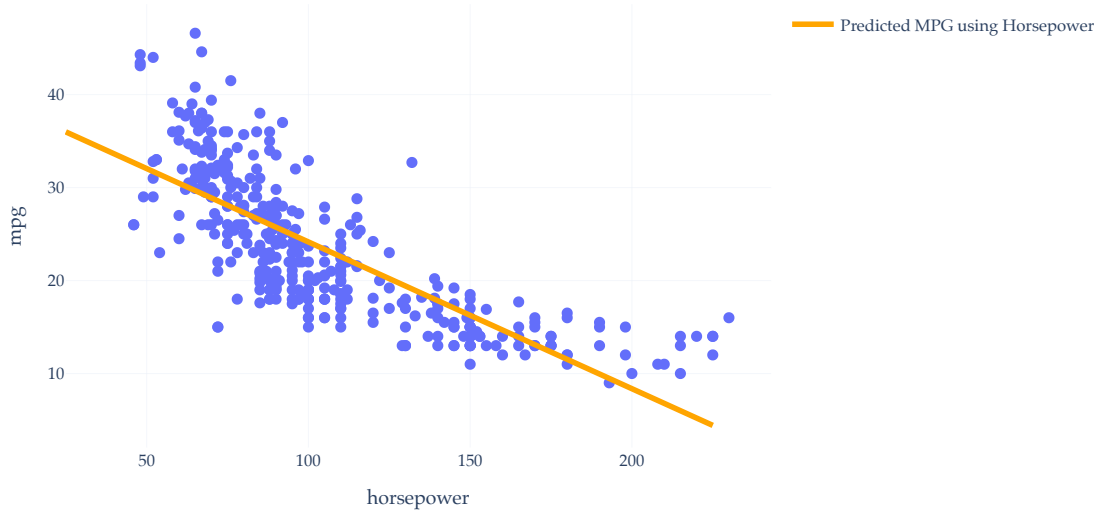
In [6]:

```
car_model = LinearRegression()
car_model.fit(mpg[['horsepower']], mpg['mpg'])
```

Out[6]:

In [7]:

```
hp_points = pd.DataFrame({'horsepower': [25, 225]})
fig = px.scatter(mpg, x='horsepower', y='mpg')
fig.add_trace(go.Scatter(
    x=hp_points['horsepower'],
    y=car_model.predict(hp_points),
    mode='lines',
    name='Predicted MPG using Horsepower',
    line={'color': 'orange', 'width': 4},
))
fig
```



- Our regression line doesn't capture the curvature in the relationship between `horsepower` and `mpg`.
- As a baseline, here's the training MSE for the model that uses raw `horsepower`.

In [8]:

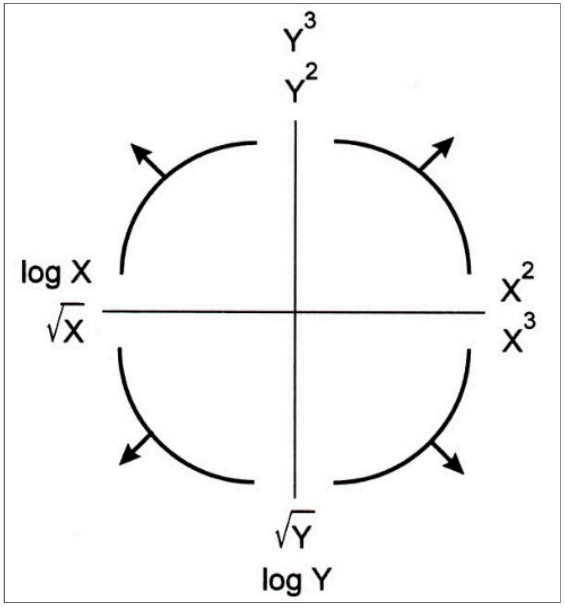
```
horsepower_mse = mean_squared_error(mpg['mpg'], car_model.predict(mpg[['horsepower']]))
horsepower_mse
```

Out[8]:

```
23.943662938603104
```

Creating a logarithmic feature

- The **Tukey Mosteller Bulge Diagram** helps us pick which **numerical-to-numerical** transformations to apply to data in order to **linearize** it.
  Alternative interpretation: it helps us determine which features to create.



- Here, the bottom-left quadrant appears to match the shape of the scatter plot between `horsepower` and `mpg` the best.

In [9]:

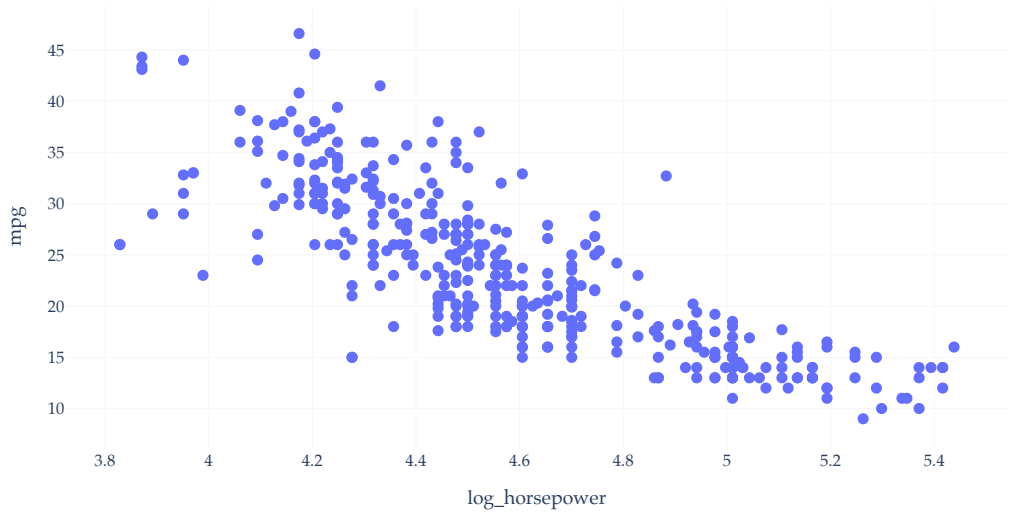
```
mpg_log_view = mpg.assign(log_horsepower=np.log(mpg['horsepower']))
mpg_log_view.head()
```

Out[9]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name	log_horsepower
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrolet chevelle malibu	4.867534
1	15.0	8	350.0	165.0	3693	11.5	70	usa	buick skylark 320	5.105945
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymouth satellite	5.010635
3	16.0	8	304.0	150.0	3433	12.0	70	usa	amc rebel sst	5.010635
4	17.0	8	302.0	140.0	3449	10.5	70	usa	ford torino	4.941642

In [10]:

```
px.scatter(mpg_log_view, x='log_horsepower', y='mpg')
```



- The resulting data looks slightly more linear than the original data.

Predicting `mpg` using `log(horsepower)`

- Let's try and predict `mpg` using `log(horsepower)`. Specifically, we're fitting the model

$$h(\text{horsepower}_i) = w_0 + w_1 \cdot \log(\text{horsepower}_i)$$

In [11]:

```
car_model_log = LinearRegression()
car_model_log.fit(X=mpg_log_view[['log_horsepower']], y=mpg_log_view['mpg'])
```

Out[11]:

LinearRegression

LinearRegression()

In [12]:

```
car_model_log.intercept_, car_model_log.coef_
```

Out[12]:

```
(108.69970699574483, array([-18.58218476]))
```

In [13]:

```
# Lower than before, though this is just training MSE.
mean_squared_error(mpg_log_view['mpg'], car_model_log.predict(mpg_log_view[['log_horsepower']]))
```

Out[13]:

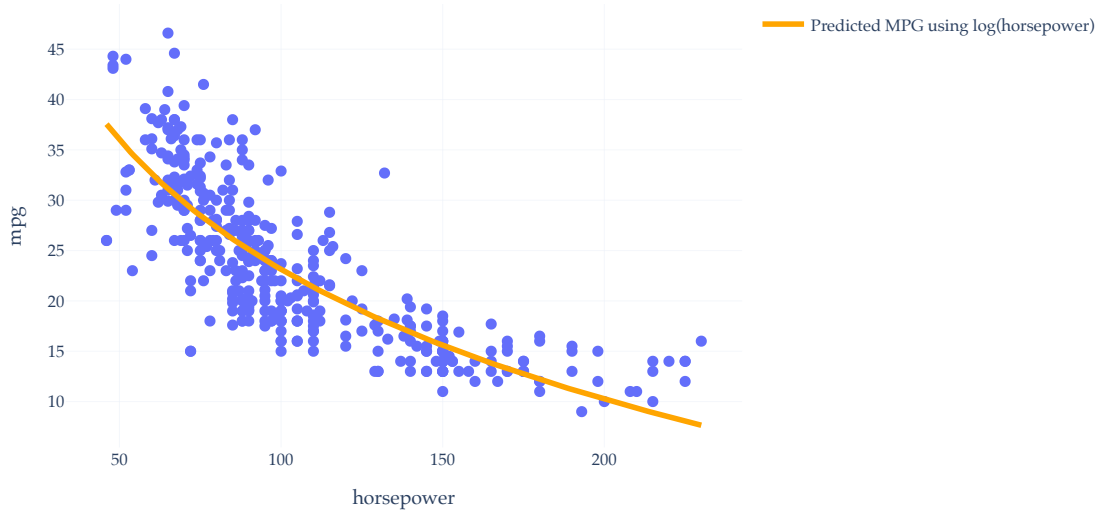
```
20.152887978233167
```

In [14]:

```
hp_points = pd.DataFrame({'horsepower': np.linspace(mpg['horsepower'].min(), mpg['horsepower'].max(), 200)})
hp_points['log_horsepower'] = np.log(hp_points['horsepower'])

fig = px.scatter(mpg, x='horsepower', y='mpg')
fig.add_trace(go.Scatter(
    x=hp_points['horsepower'],
    y=car_model_log.predict(hp_points[['log_horsepower']]),
```

```
    mode='lines',
    name='Predicted MPG using log(horsepower)',
    line={'color': 'orange', 'width': 4},
))
fig
```



Discuss

1. Despite this being a linear model (we fit it using `LinearRegression`, why does it appear non-linear?
2. What is suboptimal about our programming approach, from a reproducibility standpoint?
   This could be code that your LLM writes!

Pipelines

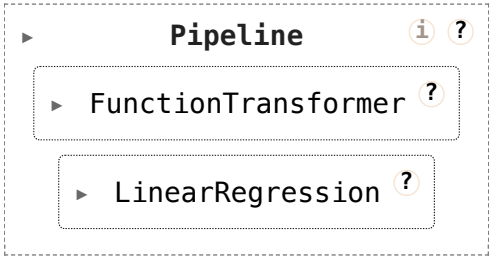
- Instead of adding new columns to our DataFrame with the transformed features, let's construct a `Pipeline` object.
- This way, the end user only needs to supply **raw data**, and the object performs the transformation.
- More sophisticated models behave the same way; when you use an LLM, you're not asked to tokenize or compute embeddings for your inputs yourself.
- Note that all of the necessary imports are in `setup.py`.

In [15]:

```
car_pipe = make_pipeline(
    FunctionTransformer(np.log),
    LinearRegression()
)

car_pipe
```

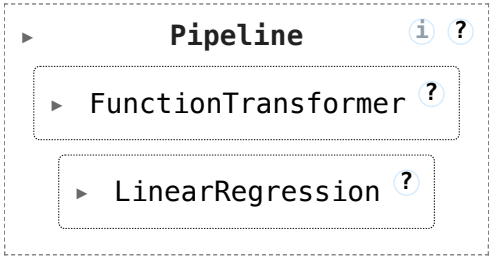
Out[15]:



In [16]:

```
car_pipe.fit(mpg[['horsepower']], mpg['mpg'])
```

Out[16]:



In [17]:

```
# Don't need to log the horsepower first!
car_pipe.predict([[150]])
```

```
/Users/surajrampure/miniforge3/envs/eecs245/lib/python3.10/site-packages/sklearn/base.py:493: UserWarn
ing:

X does not have valid feature names, but LinearRegression was fitted with feature names
```

Out[17]:

```
array([15.59115618])
```

- Conceptually, the `Pipeline` is fitting a model with this **design matrix**:

$$X = \begin{bmatrix} 1 & \log(x_1) \\ 1 & \log(x_2) \\ \vdots & \vdots \\ 1 & \log(x_n) \end{bmatrix}$$

- The important workflow detail is that future predictions also receive the same logarithmic transformation.

Example 2: Standardized coefficients

- Suppose our goal is to predict `net_sales` given the other four features.
- No transformations are *needed* to predict `net_sales`.
- Suppose we're interested in learning **how** the various features impact `net_sales`, rather than just predicting `net_sales` for a new store. We'd then look at the coefficients.

In [18]:

```
sales = pd.read_csv('data/sales.csv')
sales.head()
```

Out[18]:

	net_sales	sq_ft	inventory	advertising	district_size	competing_stores
0	231.0	3.0	294	8.2	8.2	11
1	156.0	2.2	232	6.9	4.1	12
2	10.0	0.5	149	3.0	4.3	15
3	519.0	5.5	600	12.0	16.1	1
4	437.0	4.4	567	10.6	14.1	5

In [19]:

```
sales_model = LinearRegression()
sales_model.fit(X=sales.iloc[:, 1:], y=sales.iloc[:, 0])
```

Out[19]:

▾ LinearRegression

```
LinearRegression()
```

In [20]:

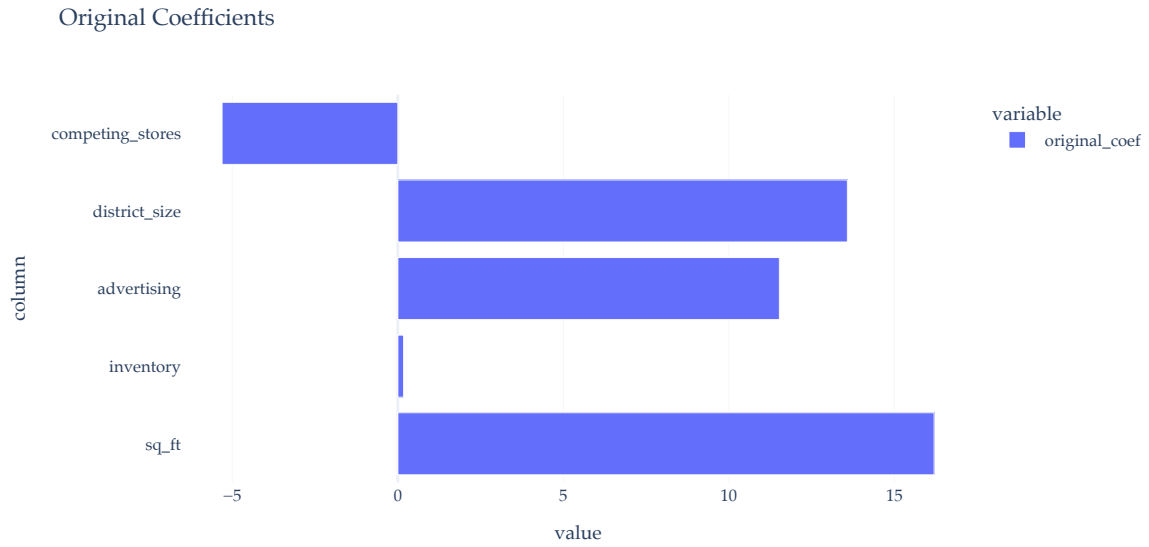
```
mean_squared_error(sales.iloc[:, 0], sales_model.predict(sales.iloc[:, 1:]))
```

Out[20]:

```
242.27445717154959
```

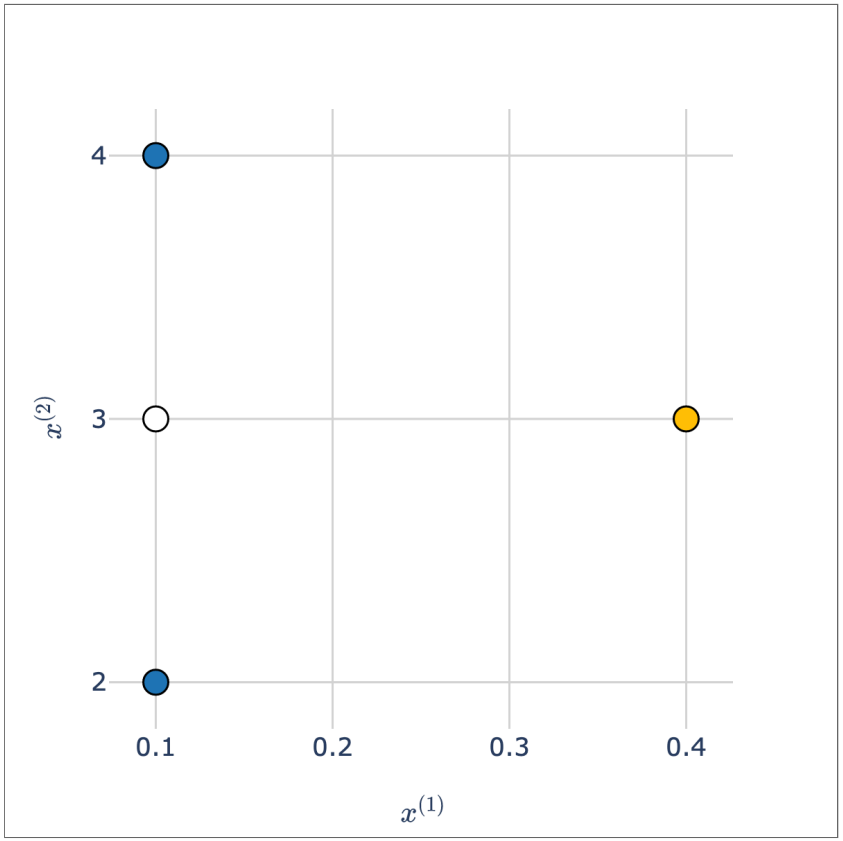
In [21]:

```
coefs = pd.DataFrame().assign(
    column=sales.columns[1:],
    original_coef=sales_model.coef_,
).set_index('column')
coefs.plot(kind='barh', title='Original Coefficients')
```


Original Coefficients
competing_stores
district_size
advertising
inventory
sq_ft
column
−5
0
5
10
15
value
variable
original_coef

Thought experiment

- Consider the white point in the scatter plot below.



- Which class is it more "similar" to – **blue** or **orange**?
- Intuitively, the answer may be **blue**, but take a close look at the scale of the axes! The **orange** point is much closer to the white point than the **blue** points are.

Standardization

- When we standardize two or more features, we bring them to the **same scale**.
- Recall: to standardize a feature $x_1, x_2, \ldots, x_n$, we use the formula:

$$z(x_i) = \frac{x_i - \bar{x}}{\sigma_x}$$

- Example: 1, 7, 7, 9.
  - Mean: $\frac{1+7+7+9}{4} = \frac{24}{4} = 6$.
  - Standard deviation:

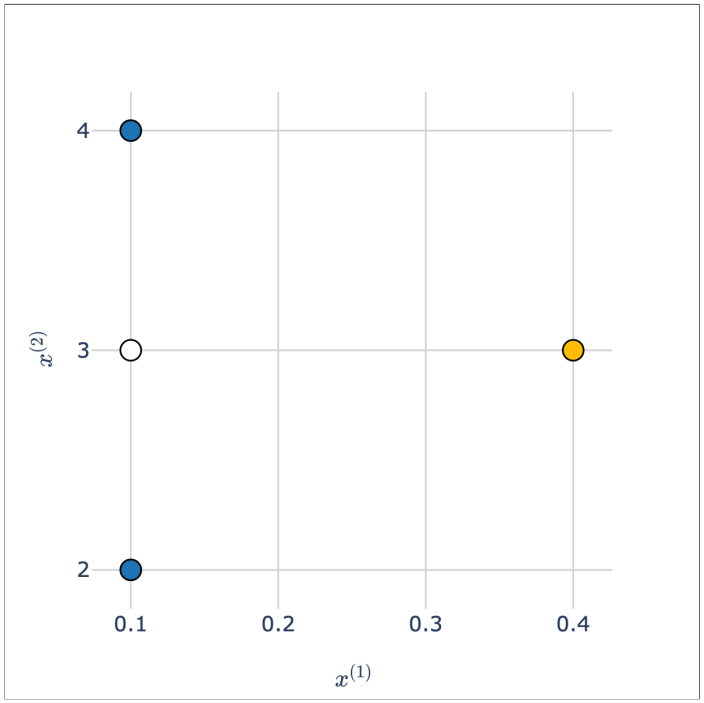
$$\mathrm{SD} = \sqrt{\frac{1}{4}\left((1-6)^2 + (7-6)^2 + (7-6)^2 + (9-6)^2\right)} = \sqrt{\frac{1}{4} \cdot 36} = 3$$

  - Standardized data:

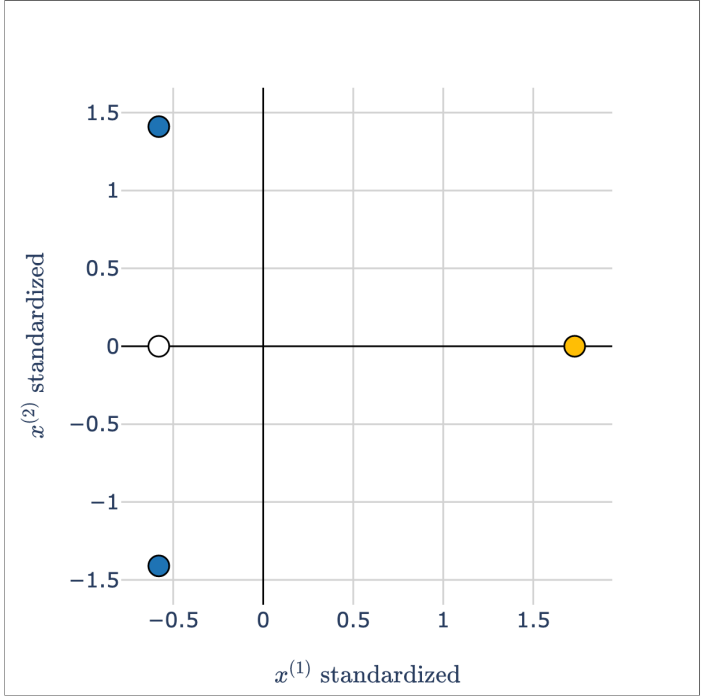
$$1 \mapsto \frac{1-6}{3} = \boxed{-\frac{5}{3}} \qquad 7 \mapsto \frac{7-6}{3} = \boxed{\frac{1}{3}} \qquad 7 \mapsto \boxed{\frac{1}{3}} \qquad 9 \mapsto \frac{9-6}{3} = \boxed{1}$$

Pre- and post- standardization

- **Before** we standardize both axes:



- **After** we standardize both axes:



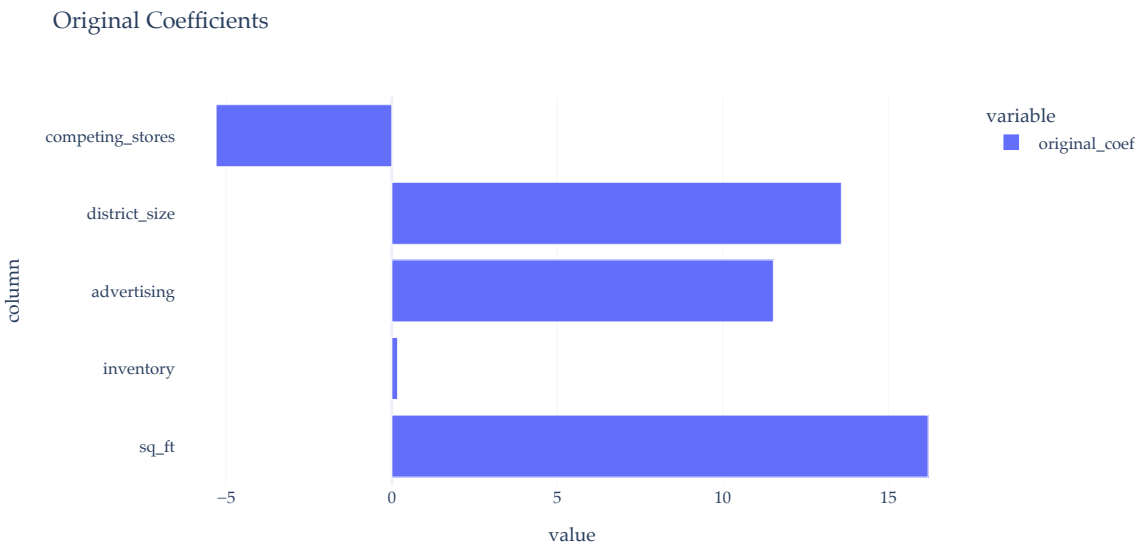
- After we standardize, our features are measured on the same scales, so **they can be compared directly**.

Which features are most "important"?

- The most important feature is **not necessarily** the feature with largest magnitude coefficient, because different features may be on different scales.
  But, it would be nice if the most important feature was the one with the largest magnitude coefficient!
- Here, `inventory` values are much larger than `sq_ft` values, which means that the coefficient for `inventory` will inherently be smaller, even if `inventory` is a more important feature than `sq_ft`.
  Intuition: if the values themselves are larger, you need to multiply them by smaller coefficients to get the same predictions!
- **Solution**: if you care about the interpretability of the resulting coefficients, **standardize** each feature before training the model.

In [22]:

```
coefs.plot(kind='barh', title='Original Coefficients')
```



In [23]:

```
sales.iloc[:, 1:].head()
```

Out[23]:

	sq_ft	inventory	advertising	district_size	competing_stores
0	3.0	294	8.2	8.2	11
1	2.2	232	6.9	4.1	12
2	0.5	149	3.0	4.3	15
3	5.5	600	12.0	16.1	1
4	4.4	567	10.6	14.1	5

Standardizing before training

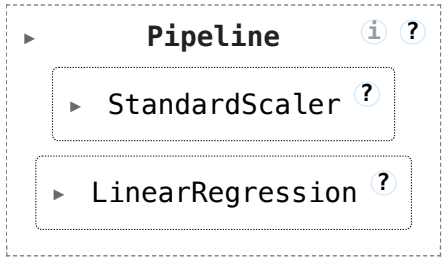
- The transformer class to use is `StandardScaler`.

In [24]:

```
sales_pipe = make_pipeline(
    StandardScaler(),
    LinearRegression()
)

sales_pipe.fit(sales.iloc[:, 1:], sales.iloc[:, 0])
```

Out[24]:



In [25]:

```
# Same as before! Why?
mean_squared_error(sales.iloc[:, 0], sales_pipe.predict(sales.iloc[:, 1:]))
```

Out[25]:

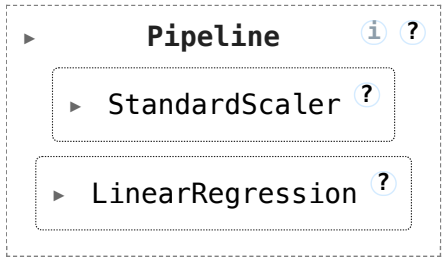
```
242.27445717154944
```

Peeking under the hood

In [26]:

```
sales_pipe
```

Out[26]:



In [27]:

```
sales_pipe[-1]
```

Out[27]:

In [28]:

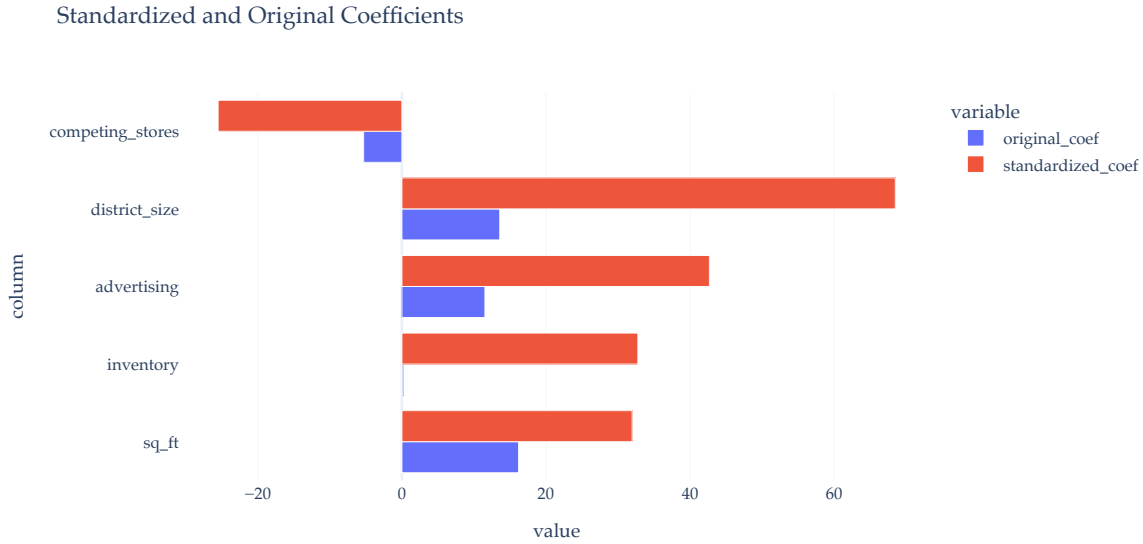
```
sales_pipe[-1].coef_
```

Out[28]:

```
array([ 31.97302867,  32.76054166,  42.69274551,  68.49841225,
       -25.51529781])
```

In [29]:

```
pd.DataFrame().assign(
    column=sales.columns[1:],
    original_coef=sales_model.coef_,
    standardized_coef=sales_pipe[-1].coef_
).set_index('column').plot(kind='barh', barmode='group', title='Standardized and Original Coefficients')
```


Standardized and Original Coefficients
competing_stores
district_size
advertising
inventory
sq_ft
column
−20
0
20
40
60
value
variable
original_coef
standardized_coef

Key takeaways

- The result of standardizing each feature (separately!) is that the units of each feature are on the same scale.
    - There's no need to standardize the outcome (`net_sales` here), since it's not being compared to anything.
- The resulting $w_0^*, w_1^*, \ldots, w_d^*$ are called the **standardized regression coefficients**.
- Standardized regression coefficients can be directly compared to one another.
  Features with larger (absolute) standardized regression coefficients influence the model's predictions more than others.
- Standardizing each feature **does not** change the MSE of the resulting model, but influences its interpretability.

Example 3: Categorical features and multiple transformations

- As a final example of feature engineering, let's return to the commute times dataset.

In [30]:

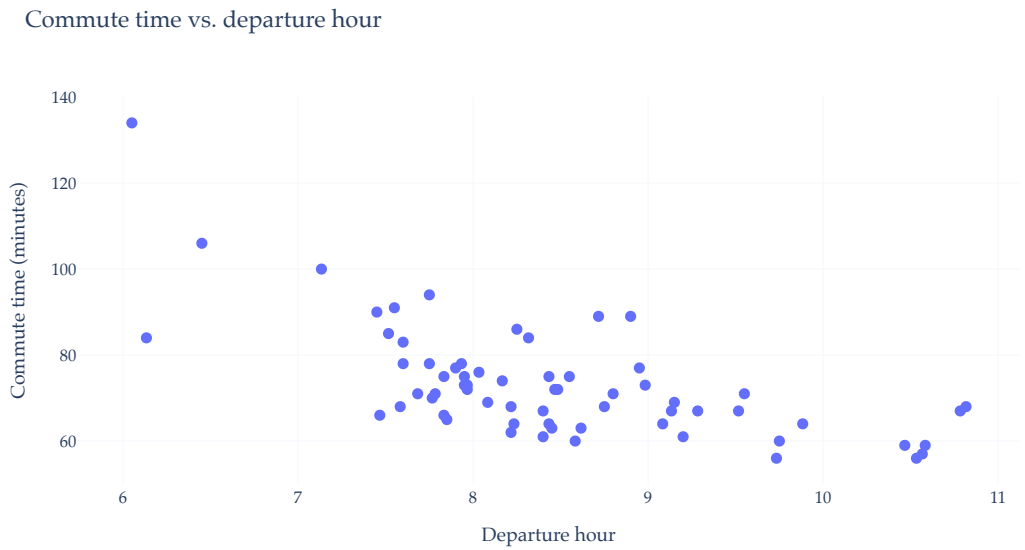
```
commutes = pd.read_csv('data/commute-times.csv')
commutes['day_of_month'] = pd.to_datetime(commutes['date']).dt.day
commutes[['departure_hour', 'day', 'month', 'day_of_month', 'minutes']].head()
```

Out[30]:

	departure_hour	day	month	day_of_month	minutes
0	10.816667	Mon	May	15	68.0
1	7.750000	Tue	May	16	94.0
2	8.450000	Mon	May	22	63.0
3	7.133333	Tue	May	23	100.0
4	9.150000	Tue	May	30	69.0

In [31]:

```
px.scatter(
    commutes,
    x='departure_hour',
    y='minutes',
    title='Commute time vs. departure hour',
    labels={
        'departure_hour': 'Departure hour',
        'minutes': 'Commute time (minutes)',
    },
    hover_data=['date', 'day'],
)
```



- Previously, we used `departure_hour` and `day_of_month` to predict commute time in `minutes`.

In [32]:

```
# For future reference.
model_multiple = LinearRegression()
model_multiple.fit(X=commutes[['departure_hour', 'day_of_month']], y=commutes['minutes'])
mean_squared_error(commutes['minutes'], model_multiple.predict(commutes[['departure_hour', 'day_of_month']]))
```

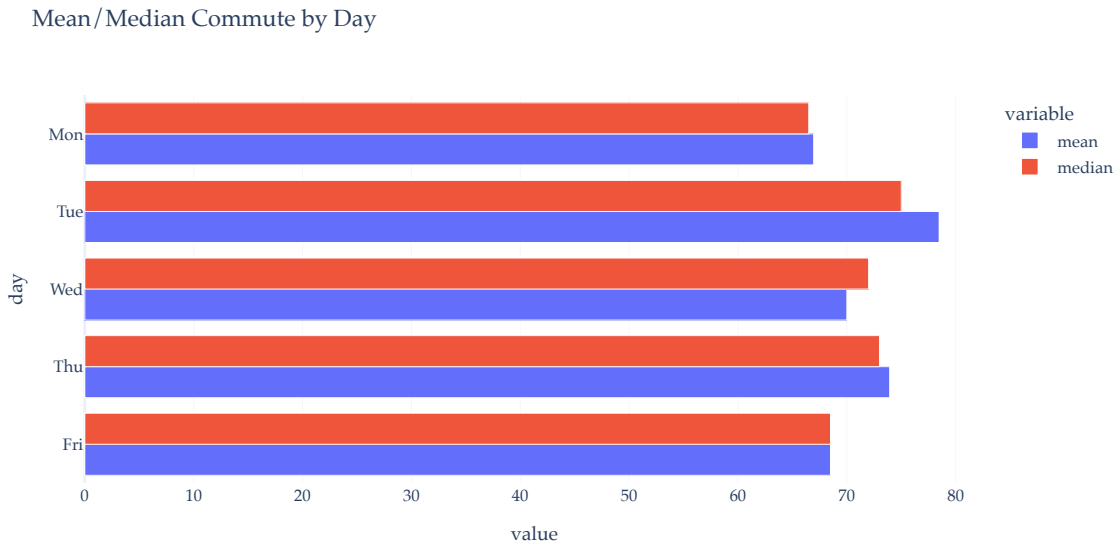
Out[32]:

```
96.78730488437492
```

Should we use `day` of the week?

In [33]:

```
(
    commutes.groupby('day')['minutes'].agg(['mean', 'median'])
    .loc[['Mon', 'Tue', 'Wed', 'Thu', 'Fri']]
    .iloc[::-1]
    .plot(kind='barh', barmode='group', title='Mean/Median Commute by Day')
)
```



One-hot encoding

- One-hot encoding is the process of turning a categorical feature into several binary features (sometimes called **dummy variables**), one per category.

day	day == Mon	day == Tue	day == Wed	day == Thu	day == Fri
Mon	1	0	0	0	0
Tue	0	1	0	0	0
Mon	1	0	0	0	0
...	...	...	...	...	...
Mon	1	0	0	0	0
Tue	0	1	0	0	0
Thu	0	0	0	1	0

- Each value of that categorical variable ends up with its own feature, and thus its own parameter.
- We drop one of the categorical features to avoid **multicollinearity**.

Using `OneHotEncoder` and `make_column_transformer`

- Unlike with previous transformations, we don't want to apply the same transformation to every single feature: it would be nonsensical to one-hot encode `departure_hour`.
- Instead, we'll need to specify that different columns need different transformations. `make_column_transformer` helps with this.

In [34]:

```
commutes[['departure_hour', 'day', 'month', 'day_of_month', 'minutes']].head()
```

Out[34]:

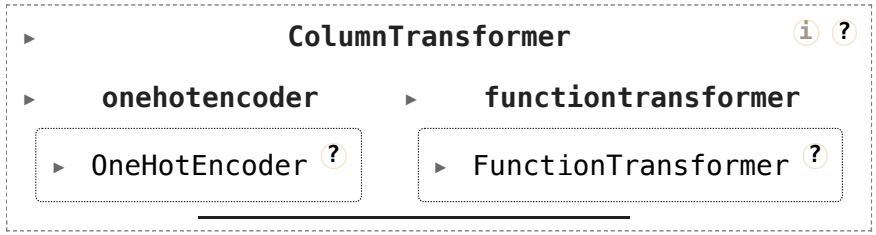
	departure_hour	day	month	day_of_month	minutes
0	10.816667	Mon	May	15	68.0
1	7.750000	Tue	May	16	94.0
2	8.450000	Mon	May	22	63.0
3	7.133333	Tue	May	23	100.0
4	9.150000	Tue	May	30	69.0

In [35]:

```
commute_trans = make_column_transformer(
    (OneHotEncoder(drop='first'), ['day']),
    (FunctionTransformer(lambda x: x), ['departure_hour', 'day_of_month']),
    remainder='drop'
)

commute_trans
```

Out[35]:

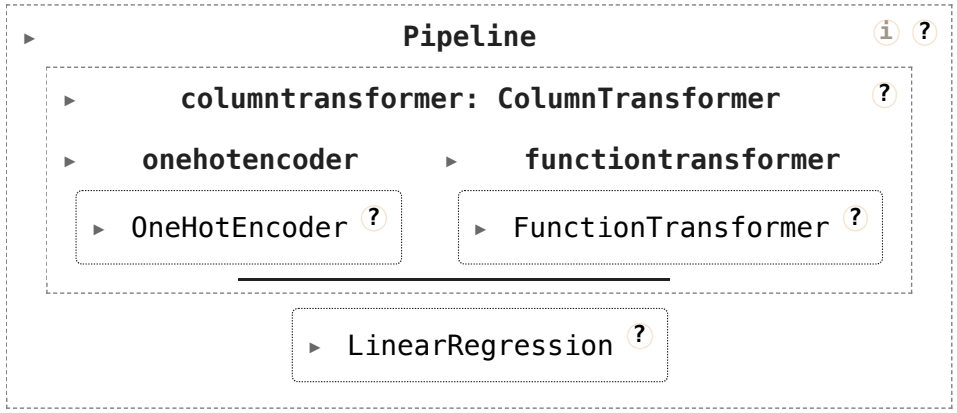


In [36]:

```
commute_pipe = make_pipeline(
    commute_trans,
    LinearRegression()
)

commute_pipe
```

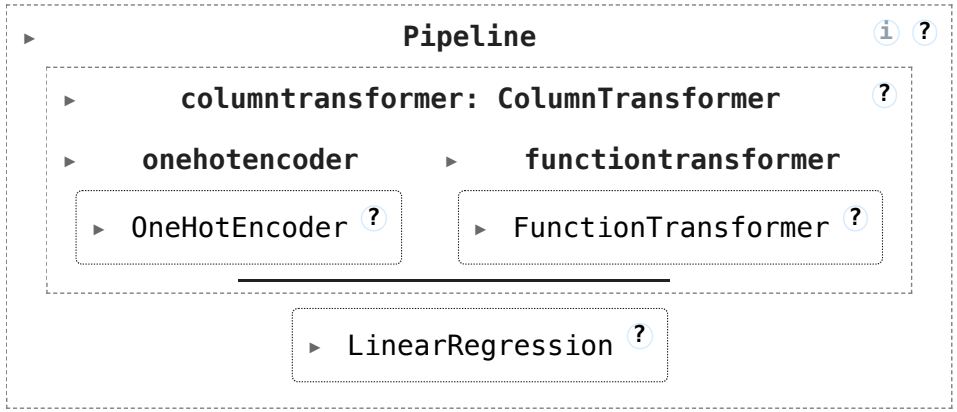
Out[36]:



In [37]:

```
commute_pipe.fit(commutes[['departure_hour', 'day_of_month', 'day']], commutes['minutes'])
```

Out[37]:



In [38]:

```
# Much lower than the ~97 from the baseline!
mean_squared_error(
    commutes['minutes'],
    commute_pipe.predict(commutes[['departure_hour', 'day_of_month', 'day']])
)
```

Out[38]:

```
70.21791287461919
```

Activity

1. How many parameters does the `commute_pipe` model have? (Think of an answer, then try and write code to answer it.)
2. (Attempt to) Create a new `Pipeline` that:
    A. Creates degree 2 polynomial features using `departure_hour`; use `PolynomialFeatures` for this. Why is this reasonable?
    B. One-hot encodes **both** `day` and `month`.
    C. Converts `day_of_month` into a week-of-month label, then one-hot encodes that; use `FunctionTransformer` for this.

This is an extreme example; in practice, we need to be able to justify our chosen features.

In [ ]:

```
```

Key idea

- The final `Pipeline` fits the feature transformations and the regression model as one reproducible workflow.
- Future predictions now use the same column selection, polynomial expansion, one-hot encoding, week conversion, and model fit.
- End users don't need to manually reproduce the feature creation steps, but have a clear depiction of the `Pipeline`.
- Reproducible and interpretable!

Example: iBudget Data

Discuss: Florida's iBudget program

Once again, open:
https://apd.myflorida.com/resources/reports/APD%20Algorithm%20Report.pdf

- How many features does the current model use?
- What is the target variable, and how was it justified?
- What feature engineering steps did they use to create them?
- Answer the questions above for the new model ("Model 9") as well.

In [ ]:

Back to the birthweight dataset 👶

Back to the birthweight data

- We have now seen several reusable transformations: logarithms, polynomial features, one-hot encoding, and standardization.
- Your job is to use the same `Pipeline` idea on the real Unit 3 birthweight data.
- **Remember**, we need to be able to justify which features we create.

For reference: Baseline model

- Here's a complete implementation of a baseline model, as you were meant to create at the end of Notebook 1.
- Unlike you may have done there, we will keep all features in the dataset, but first drop rows with missing values (a "complete case analysis" in the language of Unit 3).
- **Discuss**: why do ~50% of rows have a missing `interval` value?

In [39]:

```
births.isna().mean()
```

Out[39]:

```
year           0.000000
state          0.000000
county         0.000000
smsa           0.000000
sex            0.000000
dadrace        0.080294
momrace        0.004874
momage         0.000000
birthorder     0.013580
dadage         0.089990
birthweight    0.002558
plurality      0.000000
interval       0.582936
popsize        0.000000
dtype: float64
```

In [40]:

```
births = births.dropna()
```

In [41]:

```
X_train, X_test, y_train, y_test = train_test_split(
    births.drop(columns=['birthweight']),
    births['birthweight'],
    test_size=0.2,
    random_state=23
)
```

In [42]:

```
features = ['momage', 'dadage', 'plurality', 'birthorder']
baseline = LinearRegression()
baseline.fit(X_train[features], y_train)
```

Out[42]:

LinearRegression

LinearRegression()

In [43]:

```
# Train MSE.
mean_squared_error(y_train, baseline.predict(X_train[features]))
```

Out[43]:

329824.89474364807

In [44]:

```
# Test MSE.
mean_squared_error(y_test, baseline.predict(X_test[features]))
```

Out[44]:

329866.361889481

Activity: Fit an improved model

Here are your tasks:

1. Explore the `births` DataFrame and think of 2-3 new features to create, remembering that `birthweight` is the target variable and that chosen features should be justified. **You're encouraged to use LLMs to generate exploratory code (visuals, etc.).**
2. Construct a `Pipeline` object that can be called as follows:

```
pipe.fit(X_train, y_train)
```

3. Compare your new `Pipeline`'s performance on the **test set** to your old baseline model's performance on the test set.

Note that it's important that we use the **same train/test split** in both the baseline and updated model; otherwise we're not comparing apples to apples.

In [ ]:

```
```

Some ideas...

- Was the birth a "single" birth or "multiple" birth?
- What is the age gap of the parents?
- Does parental race play a role?
- What is the "age bin" of the mother? Why could this be better or worse than using `momage` directly? (Look into `pd.cut`.)

In [45]:

```
# Remember NOT to reassign the DataFrame.
# If you want to use `is_multiple` as a feature,
# it must be in your Pipeline!
(
    births
    .assign(is_multiple=births['plurality'] > 1)
    .groupby('is_multiple')
    ['birthweight']
    .agg(['mean', 'median', 'std', 'count'])
)
```

Out[45]:

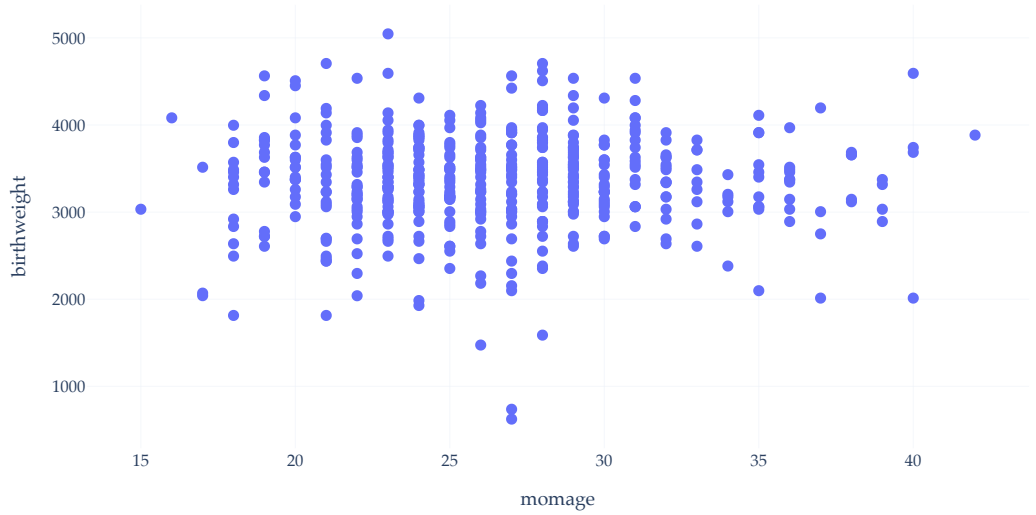
is_multiple	mean	median	std	count
False	3361.117265	3374.0	573.237981	680857
True	2426.138728	2495.0	661.170621	17055

Example visualizations

- Recall, `births` has over a million rows – using all of them to create visualizations is taxing.
- Run the cells below for some example visuals.

In [46]:

```
births.sample(500).plot(kind='scatter', x='momage', y='birthweight')
```



In [47]:

```
births.sample(500).plot(kind='box', x='sex', y='birthweight')
```


5000
4000
3000
2000
1000
birthweight
male
female
sex

Deliverables

To ALICE, include:

1. A screenshot of your final model's rendered `Pipeline` diagram.
2. A plot or table that motivated one of your features.
3. One sentence explaining a new feature you created and why you created it.
4. One feature that you created that might be unmotivated or redundant.